

Vibe Coding Academy Live Call Summary

Bootcamp #17

June 5, 2026

Hermes Desktop, Local AI Workflows, Agent Orchestration, Q&A

Overview

This session focused on:

- New features in Hermes Desktop
 - Profiles vs. Sub-Agents
 - Local AI use cases
 - Building automated opportunity-finding systems
 - Hermes vs. OpenClaw
 - Agent memory systems
 - Local hardware recommendations
 - Agent orchestration challenges
 - Community Q&A
-

Hermes Desktop Overview

Alex opened the session by reviewing the newly released Hermes Desktop application.

Why Hermes Desktop Matters

The desktop app provides a significantly better experience than using Hermes exclusively through Telegram.

Key improvements include:

Session Management

Instead of maintaining one giant conversation thread, users can now create separate sessions for specific topics:

Examples:

- Content Creation
- Local AI
- Stock Research
- Coding
- Business Strategy

Benefits:

- Cleaner context
- Better responses
- Lower token costs
- Faster performance

Alex recommends creating separate sessions for every major area of work.

Artifacts

Hermes Desktop automatically organizes:

- Links
- Images
- Files
- Media

A practical workflow is sending links to Hermes throughout the day and later retrieving them from the organized artifacts panel.

Skills & Tools Management

Many users unknowingly have hundreds of skills enabled.

Every enabled skill adds context overhead because Hermes evaluates whether each skill may be relevant to every message.

Benefits of disabling unused skills:

- Reduced token usage
- Faster responses
- Improved quality

Recommendation:

Review enabled skills regularly and disable anything not currently needed.

Profiles

Profiles allow users to create multiple specialized Hermes agents.

Examples Alex uses:

GPT Profile

- Coding tasks
- Coding memories
- Coding skills

Quen Profile

- Research
- Web scraping
- Signal gathering

Each profile maintains:

- Its own memories
- Skills
- Context
- Sessions
- Personality

This keeps agents lean and specialized.

Profiles vs. Sub-Agents

A common question was the difference between Profiles and Sub-Agents.

Sub-Agents

Sub-agents are copies of the primary agent.

Example:

"Build me an app."

Hermes may create multiple sub-agents to work on different parts of the application.

All sub-agents share:

- Skills
 - Memories
 - Context
 - Capabilities
-

Profiles

Profiles are separate specialists.

Example video workflow:

Research Agent

→ Researches topic

Writing Agent

→ Creates script

Image Agent

→ Creates thumbnail

Social Agent

→ Creates tweets

These should be separate profiles because they require different capabilities.

Cron Jobs

Hermes Desktop includes a visual Cron Job interface.

Users can:

- Create recurring tasks
- Verify tasks were created
- Edit schedules
- Review active automations

This solves a common problem where users weren't sure whether scheduled tasks were actually created.

Examples:

- Daily reports
 - Morning summaries
 - Evening reviews
 - Scheduled research tasks
-

Local AI Use Case: Finding Business Opportunities

Alex demonstrated a local AI workflow running on a DGX Spark using Quen 36.

Every 20 minutes:

The system:

1. Scrapes Reddit
2. Scrapes X/Twitter
3. Identifies complaints
4. Finds unmet needs
5. Generates business opportunities
6. Builds prototype concepts

Example:

The system discovered people wanted a better AI influencer discovery and outreach platform.

The agent then:

- Identified the opportunity
 - Explained why Alex was positioned to solve it
 - Generated a prototype concept
-

Why Local Models Matter

Alex emphasized:

Local models are not necessarily:

- Smarter
- Faster

Their advantage is:

24/7 Operation

They can run continuously with effectively zero usage costs beyond electricity.

This enables workflows that would become prohibitively expensive using cloud APIs.

Hermes vs. OpenClaw

Alex's current preference:

Hermes

Reasons:

- More focused product vision
- More reliable releases
- Better user experience
- New desktop app

However, Alex emphasized:

If OpenClaw is working for you, there is no reason to switch.

Both tools can coexist.

There is no need for tribalism.

Model-Based Profiles

Alex explained that he no longer organizes agents according to human business roles.

Instead of:

- CEO
- CTO
- Product Manager
- Developer

He prefers organizing around model strengths.

Examples:

- GPT-5.5 profile
- Opus profile
- Qwen profile

Reason:

Models already possess broad intelligence.

Assigning rigid human organizational structures often creates unnecessary complexity.

Shared Memory Systems

Question:

How should multiple agents share memory?

Alex's approach:

Individual Memory

Each agent has its own memory.

Shared Memory Layer

Agents also reference a common memory directory.

His current implementation uses Obsidian.

Shared files contain:

- Important facts
- Project knowledge
- Cross-agent context

Agents can both read and update shared memory when appropriate.

Agent Orchestration Discussion

A member described challenges using Linear for project orchestration.

Problem:

Tasks become stuck during execution.

Alex suggested implementing a monitoring framework similar to the "Ralph Loop."

Concept:

A supervisory agent periodically reviews:

- Progress
- Completion status
- Goal alignment

This creates a feedback loop that helps prevent agents from stalling.

Hardware Recommendations

Alex cautioned against buying expensive hardware without first identifying use cases.

Recommended process:

Step 1

Document:

- Business processes
- Team responsibilities
- Existing workflows

Step 2

Use AI to identify:

- Automation opportunities
- Repetitive tasks
- Tasks suitable for local models

Step 3

Determine:

- Which tasks require cloud models
- Which tasks can run locally

Step 4

Purchase hardware based on actual needs.

Not the other way around.

Local Model Recommendations

Recommended range:

27B–35B Models

Benefits:

- Good intelligence
- Good speed
- Reasonable hardware requirements

Examples:

- Qwen 27B
- Similar mid-sized models

Alex considers this the current sweet spot for most users.

Community Guidelines Reminder

A question arose regarding offering paid consulting services to fellow community members.

Alex clarified:

Members are welcome to:

- Help each other
- Answer questions
- Share knowledge

Members should not:

- Solicit services
- Send sales pitches
- Use the community primarily for lead generation

The purpose of the community is learning and collaboration.

Key Takeaways

1. Download and use Hermes Desktop.
2. Organize work using Sessions and Profiles.
3. Disable unused Skills to reduce token usage.
4. Use local models for long-running automated workflows.
5. Build agents around model strengths rather than business titles.
6. Implement shared memory systems when multiple agents collaborate.
7. Use monitoring systems to prevent agents from getting stuck.
8. Define automation opportunities before purchasing hardware.
9. Focus on solving real user problems identified through signal gathering.
10. Keep community interactions educational rather than sales-focused.